Lab 6 Code and Branch Coverage Analysis

Final Laboratory Report

This report demonstrates how complete line execution can still leave decision outcomes untested. The final test suites achieve 100% line coverage and 100% branch coverage for the discount calculator, exceeding the minimum 90% requirement.

Student Name: _____

Student ID: _____

Course: _____

Instructor: _____

Submission Date: 1 November 2026

Objective

The objective was to use Python coverage analysis to distinguish statement execution from decision-outcome execution, identify untested branches, and design targeted tests that raise both metrics above 90%.

Software and Project Structure

Component	Selection
Language	Python 3
Test framework	pytest 8.4.2
Coverage tool	coverage.py 7.11.0
Application	calculator_application.py
Line test suite	CodeCoverageTest_student_application.py
Branch test suite	BranchCoverageTest_student_application.py

Application Under Test

The calculator applies a 10% discount to members and an additional 5% discount when the price is greater than 100. The two independent if statements create four branch outcomes: member true or false, and price above 100 true or false.

```
def calculate_discount(price, is_member):
    discount = 0
    if is_member:
        discount = 0.10
    if price > 100:
        discount += 0.05
    return price * (1 - discount)
```

Coverage Method

- Run the line-focused suite with coverage collection enabled and inspect the missing-line report.
- Re-run the same single test with branch tracking to reveal decision outcomes that line coverage misses.
- Add boundary and membership combinations, then run branch coverage again.
- Accept the result only after the coverage report exceeds 90% and all tests pass.

Code Coverage Analysis

The line-coverage suite uses a member purchasing an item priced at 150. This one execution enters both if bodies, so all seven executable statements run at least once. The report therefore shows 100% line coverage with no missing statements.

Command

```
coverage run --source=calculator_application -m pytest -q
CodeCoverageTest_student_application.py
coverage report -m
```

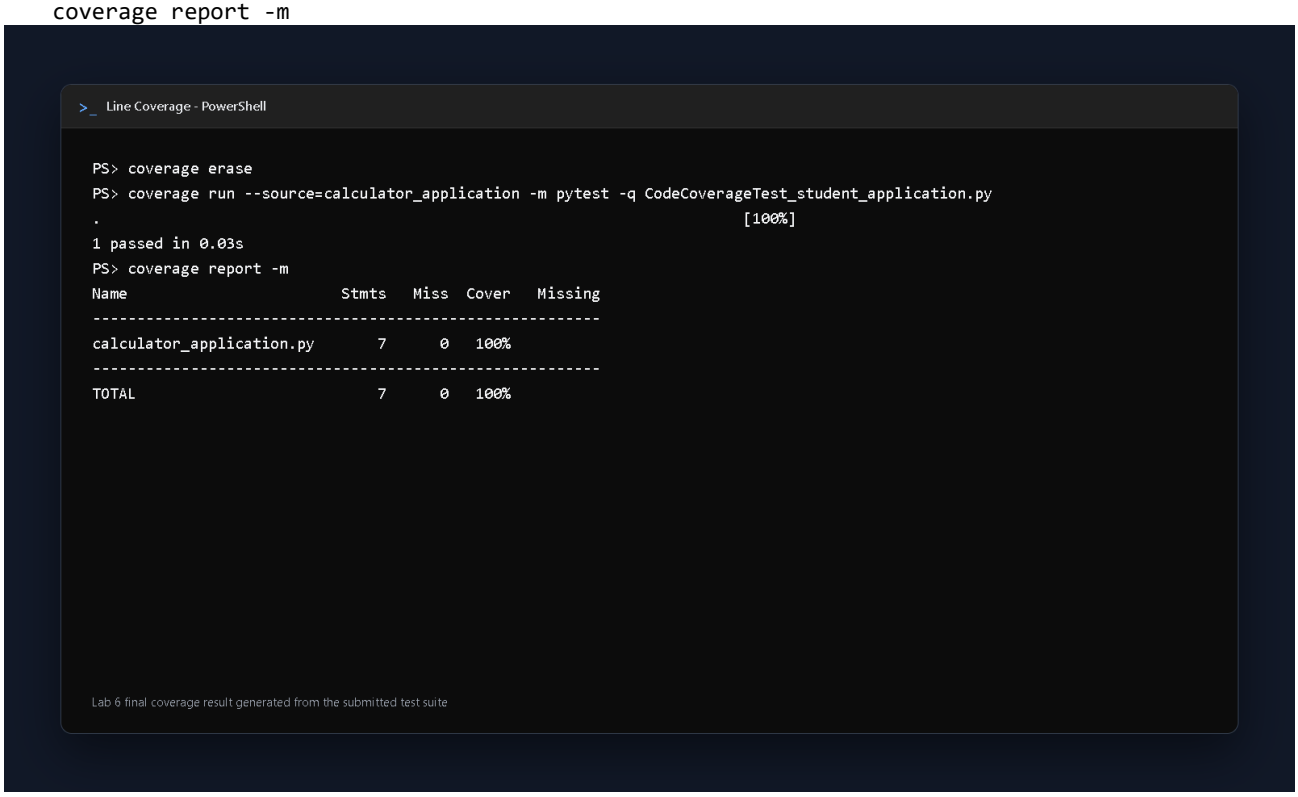


Figure 1 Final line coverage result showing 7 of 7 statements executed

Result: 1 test passed and statement coverage reached 100%. This satisfies the lab's minimum code-coverage requirement.

Why Line Coverage Was Not Sufficient

When branch tracking was enabled for the same single test, the tool reported 82% overall coverage and two partial branches. The untested outcomes were the path where is_member is False and the path where price is 100 or less. This confirms that executing every statement does not prove that every decision outcome was tested.

Branch Coverage Analysis

The branch suite uses four parameterized cases. Together they exercise True and False outcomes for both independent decisions, including the exact boundary value of 100.

Case	Price	Member	Expected	Branches
Member expensive	150	True	127.5	T, T
Member boundary	100	True	90.0	T, F
Nonmember expensive	150	False	142.5	F, T
Nonmember boundary	100	False	100.0	F, F

Command

```
coverage run --branch --source=calculator_application -m pytest -q
BranchCoverageTest_student_application.py
coverage report -m
```

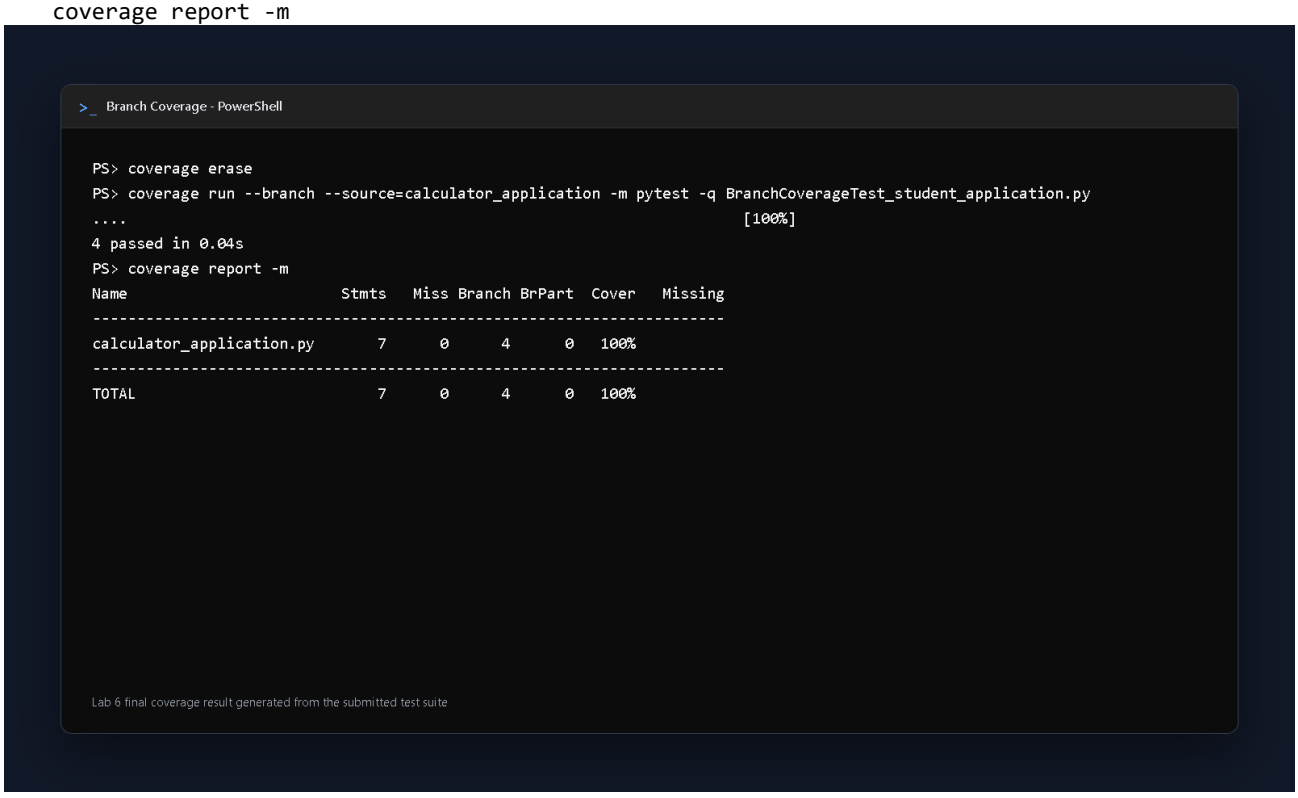


Figure 2 Final branch coverage result showing all 4 branch outcomes covered

Result: all 4 parameterized tests passed. The report shows 7 statements, 4 branches, 0 partial branches, and 100% overall coverage.

Results and Interpretation

Run	Tests	Statements	Branches	Overall
Single line-focused test	1	100%	2 partial	82% branch run
Final line coverage	1	100%	Not measured	100% line
Final branch coverage	4	100%	100%	100%

The 18 percentage-point difference between the first branch run and its 100% line result is the central finding. Line coverage answered whether each statement executed; branch coverage answered whether each decision produced both outcomes. The final suite closes the gap by testing membership status and the price boundary independently.

Repository and Commit Evidence

Public repository: https://github.com/Adi-2936/lab-6-code-branch-coverage

Commit	Purpose	Evidence
476e488	Add line coverage tests and evidence	100% line
c050faa	Add branch coverage tests and evidence	100% branch

Conclusion

The lab objectives were achieved. The line-focused test executed all statements, while the branch analysis exposed two untested False outcomes. Four targeted cases then covered all combinations of the two decisions, including the price boundary at 100. The submitted project reaches 100% line coverage and 100% branch coverage, includes final terminal-style evidence, and records the required work in separate Git commits.